

Assessing LLM-Based Code Evaluation

Euibin Bae, Joonyeong Noh, Yeseung Lee, Jooyong Yi

Generated Code

이런 json format list에서, "score" key의 값이 90보다 작은 원소가 없는 파일들의 파일명만을 리스트로 반환하고 이를 "mutant_result/1002/doesn't_dropped.txt"에 저장하는 파이썬 코드를 작성해 주실 수 있나요?

물론입니다. 아래는 요청하신 동작을 수행하는 완전한 Python 코드입니다 —

mutant_result/1002/scipy/ 하위의 모든 *.json 파일 중, 모든 항목의 "score" 가 90 이상인 파일만을 찾아 그 파일명(예: 123.json)을 "mutant_result/1002/doesn't_dropped.txt"에 저장합니다.

Python



실행됨

```
import os
import json

base_dir = "mutant_result/1002/scipy"
output_path = "mutant_result/1002/doesn't_dropped.txt"

valid_files = []

# 디렉터리 순회
for root, _, files in os.walk(base_dir):
    for file in files:
        if file.endswith(".json") and file[:-5].isdigit(): # {number}.json 형식만
            file_path = os.path.join(root, file)
            try:
                with open(file_path, "r", encoding="utf-8") as f:
                    data = json.load(f)
                # 모든 score가 90 이상인지 검사
                if all(item.get("score", 0) >= 90 for item in data):
                    valid_files.append(file)
            except Exception as e:
                print(f"⚠️ {file_path} 읽기 오류: {e}")
```

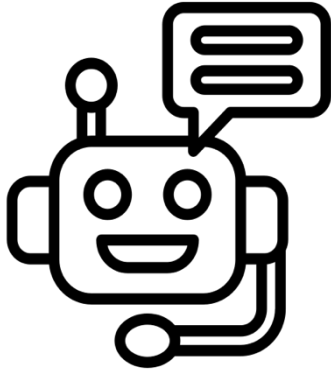
Then, can I trust it?

...

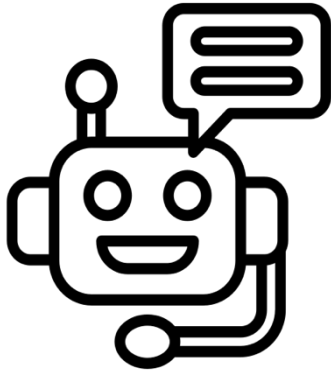


LLM-as-Judge

- Goal:



Oh, the correctness score is 100!



Oh, the correctness score is 0!



LLM-as-Judge

- Extreme Cases

```
"task_id": "99",  
"141": "pass\n",  
"grade-141": {  
    "execution": 0,  
},  
"20": "if value[-1] == \"5\":\n        return round(float(value))\n ...  
"grade-20": {  
    "execution": 0,  
},
```

LLM-as-Judge

- Extreme Cases

```
"task_id": "99",  
"141": "pass\n",  
"grade-141": {  
    "execution": 0,  
    "execution_ratio": 0.0  
},  
"20": "if value[-1] == \"5\":\n        return round(float(value))\n ...  
"grade-20": {  
    "execution": 0,  
    "execution_ratio": 0.9916083916083916  
},
```

Re-Assessment

Method	r	r_b	$r - r_b$
BLEU	0.382 / 0.272	0.406 / 0.279	-0.023 / -0.007
ROUGE-L	0.406 / 0.269	0.408 / 0.265	-0.002 / +0.004
METEOR	0.440 / 0.347	0.465 / 0.355	-0.025 / -0.008
chrF	0.416 / 0.324	0.435 / 0.332	-0.019 / -0.008
CodeBLEU	0.409 / 0.227	0.454 / 0.253	-0.045 / -0.026
RUBY	0.399 / 0.326	0.420 / 0.302	-0.021 / +0.024
CBS _{F₁}	0.407 / 0.333	0.397 / 0.284	+0.009 / +0.048
CBS _{F₃}	0.428 / 0.343	0.433 / 0.308	-0.005 / +0.035
ICE-Score	0.456 / 0.274	0.436 / 0.291	+0.020 / -0.017
ICE-Score w/o r	0.380 / 0.328	0.310 / 0.278	+0.069 / +0.050
CJ-BIN	0.575 / 0.370	0.617 / 0.397	-0.041 / -0.026
CJ-BIN w/o r	0.508 / 0.362	0.509 / 0.361	-0.001 / +0.002
CJ-SCORE	0.516 / 0.325	0.528 / 0.316	-0.012 / +0.009
CJ-SCORE w/o r	0.417 / 0.300	0.393 / 0.229	+0.024 / +0.071

Re-Assessment

Method	r	r_b	$r - r_b$
BLEU	0.382 / 0.272	0.406 / 0.279	-0.023 / -0.007
ROUGE-L	0.406 / 0.269	0.408 / 0.265	-0.002 / +0.004
METEOR	0.440 / 0.347	0.465 / 0.355	-0.025 / -0.008
chrF	0.416 / 0.324	0.435 / 0.332	-0.019 / -0.008
CodeBLEU	0.409 / 0.227	0.454 / 0.253	-0.045 / -0.026
RUBY	0.399 / 0.326	0.420 / 0.302	-0.021 / +0.024
CBS _{F1}	0.407 / 0.333	0.397 / 0.284	+0.009 / +0.048
CBS _{F3}	0.428 / 0.343	0.433 / 0.308	-0.005 / +0.035
ICE-Score	0.456 / 0.274	0.436 / 0.291	+0.020 / -0.017
ICE-Score w/o r	0.380 / 0.328	0.310 / 0.278	+0.069 / +0.050
CJ-BIN	0.575 / 0.370	0.617 / 0.397	-0.041 / -0.026
CJ-BIN w/o r	0.508 / 0.362	0.509 / 0.361	-0.001 / +0.002
CJ-SCORE	0.516 / 0.325	0.528 / 0.316	-0.012 / +0.009
CJ-SCORE w/o r	0.417 / 0.300	0.393 / 0.229	+0.024 / +0.071

Future works

- Ratio-based evaluation quantifies how often predictions are incorrect.
- ->As evaluation becomes more fine-grained, natural language specifications must also become more precise.

Thank you!